

Computer Networking

Chapter 2: Application Layer

Questions & Answers

Question 1

Performance Analysis Problem

Consider a scenario where:

- Average object size = 1 Mbit
- Request rate = 15 requests/sec
- LAN bandwidth = 100 Mbps
- Access link bandwidth = 15 Mbps
- Internet delay $\approx$ 2 seconds

Compute and explain:

1. The traffic intensity on both the LAN and access link.
2. Why response time without caching is unacceptable.
3. The effect of upgrading the access link to 100 Mbps.
4. The average response time with a caching hit rate of 0.4.

Answer

(a) Traffic Intensity

$$\text{LAN Intensity} = \frac{15 \times 1}{100} = 0.15$$

$$\text{Access Link Intensity} = \frac{15 \times 1}{15} = 1.0$$

Interpretation: The access link is fully utilized (100%), causing severe queuing and long delays.

(b) Without Caching

- LAN delay: negligible due to low utilization.

- Access link delay: extremely high since utilization $\rightarrow 1$.

Hence, the total response time becomes **unacceptably large** (possibly several seconds or minutes).

(c) Upgrading Access Link to 100 Mbps

$$\text{New Intensity} = \frac{15 \times 1}{100} = 0.15$$

Now the delay due to access link becomes negligible, so total delay $\approx$ Internet delay ≈ 2 s.

However, upgrading the link is an expensive solution.

(d) With Web Caching (Hit Rate = 0.4)

$$\begin{aligned} \text{Average Delay} &= (0.4)(0.01 \text{ s}) + (0.6)(2.01 \text{ s}) \\ &= 0.004 + 1.206 = \boxed{1.21 \text{ s}} \end{aligned}$$

Conclusion: Web caching reduces response time from several seconds to about **1.2 seconds**, providing a significant performance improvement at low cost.

Question 2

List five nonproprietary Internet applications and the application-layer protocols that they use.

Answer

Application	Application-Layer Protocol
World Wide Web (Web Browsing)	HTTP / HTTPS
E-mail	SMTP, POP3, IMAP
File Transfer	FTP
Remote Terminal Access	Telnet / SSH
Domain Name Service	DNS

Summary: These are open (nonproprietary) Internet applications standardized by the IETF. They allow interoperability among systems developed by different organizations.

Question 3

What is the difference between network architecture and application architecture?

Answer

Network Architecture: Refers to the organization of the communication process into layers — for example, the five-layer Internet architecture (Application, Transport, Network, Link, and Physical layers). It defines how data is transmitted and routed between devices across networks.

Application Architecture: Designed by the application developer and dictates the broad structure of the application itself — for example, *client-server* or *peer-to-peer (P2P)* models. It focuses on how different components of an application interact and exchange data over the network.

In summary: Network architecture structures *how networks communicate*, while application architecture structures *how applications are built to use the network*.

Question 4

For a communication session between a pair of processes, which process is the client and which is the server?

Answer

The **client** is the process that *initiates* communication by sending a request for service. The **server** is the process that *waits to be contacted* and responds to the client's request by providing the desired service.

Example: In a web transaction:

- The web browser acts as the **client**.
- The web server (such as Apache or Nginx) acts as the **server**.

Thus, the role of client or server is determined by *who initiates* and *who responds* in the communication session.

Question 5

For a P2P file-sharing application, do you agree with the statement, “There is no notion of client and server sides of a communication session”? Why or why not?

Answer

Answer: No, not entirely. In a **peer-to-peer (P2P)** file-sharing application, while there is no permanent distinction between clients and servers, the roles can still be

defined *temporarily* during a communication session.

Explanation:

- The peer that is **receiving** a file acts as the **client**, since it initiates the request.
- The peer that is **sending** the file acts as the **server**, since it provides the requested data.
- However, these roles are not fixed — the same peer may act as a client in one session and as a server in another.

Conclusion: Although P2P systems are symmetric in design, each individual file transfer can still be viewed as a temporary **client-server interaction**.

Question 6

What information is used by a process running on one host to identify a process running on another host?

Answer

A process running on one host identifies a process running on another host using a combination of two key pieces of information:

1. **IP Address** – Identifies the destination host in the network.
2. **Port Number** – Identifies the specific process (or application) running on that host.

Together, the pair (**IP address, port number**) uniquely identifies a network process. This combination is often referred to as a **socket**.

Example: If a web browser on Host A connects to a web server on Host B, it uses:

$$(\text{IP}_B, \text{Port}_{80})$$

Here, Port 80 identifies the HTTP server process on Host B.

Question 7

Suppose you wanted to do a transaction from a remote client to a server as fast as possible. Would you use UDP or TCP? Why?

Answer

Answer: To achieve the fastest possible transaction between a remote client and a server, you would use **UDP (User Datagram Protocol)**.

Reasons:

- UDP is a *connectionless* protocol — it does not require a three-way handshake like TCP, reducing setup delay.
- It provides minimal transmission overhead since there is no congestion control, flow control, or acknowledgment mechanism.
- Data can be sent immediately without waiting for connection establishment or retransmissions.

Trade-off: UDP offers higher speed but **no reliability or guaranteed delivery**. If the application can tolerate occasional data loss (e.g., DNS queries, streaming, VoIP), UDP is ideal.

In summary: Use **UDP** when low latency is more important than reliability; use **TCP** when reliable delivery is essential.

Question 8

Can you conceive of an application that requires **no data loss** and that is also **highly time-sensitive**?

Answer

Answer: Yes. One example is a **remote robotic surgery system** (telesurgery).

Explanation:

- Such systems must transmit data without any loss, since lost packets could lead to incorrect instrument movement.
- They are highly time-sensitive because even small delays can compromise precision and safety.

Other examples:

- Real-time industrial automation or control systems.
- Autonomous vehicle coordination for collision avoidance.

Question 9

List the four broad classes of services that a transport protocol can provide. For each of the service classes, indicate if either UDP or TCP (or both) provides such a service.

Answer

Answer:

The four broad classes of services provided by a transport protocol are:

1. **Reliable Data Transfer:** Ensures that data is delivered correctly and in order. **Provided by: TCP only**
2. **Flow Control:** Ensures that the sender does not overwhelm the receiver with too much data. **Provided by: TCP only**
3. **Connection-Oriented Communication:** Establishes a logical connection between sender and receiver before data transfer. **Provided by: TCP only**
4. **Multiplexing / Demultiplexing:** Allows multiple applications to share a single transport-layer protocol using port numbers. **Provided by: Both TCP and UDP**

Summary Table:

Service	TCP	UDP
Reliable Data Transfer	Yes	No
Flow Control	Yes	No
Connection-Oriented Communication	Yes	No
Multiplexing / Demultiplexing	Yes	Yes

Question 10

Recall that TCP can be enhanced with TLS to provide process-to-process security services, including encryption. Does TLS operate at the transport layer or the application layer? If the application developer wants TCP to be enhanced with TLS, what does the developer have to do?

Answer

Answer:

1. **TLS Layer:** TLS (Transport Layer Security) operates **between the transport and application layers**, often considered part of the **application layer** from the perspective of the OSI model. It enhances TCP by providing encryption, integrity, and authentication for application data.

2. **How a Developer Uses TLS:** To use TLS over TCP, the application developer must:

- Use a **TLS library or API** (e.g., OpenSSL, Java Secure Socket Extension) in their application.
- Establish a **TLS session** over a TCP connection rather than using plain TCP sockets.

- Handle certificate management (server certificate, optionally client certificate) as required for authentication.

Summary: The developer does not modify TCP itself; they build their application to use TCP *wrapped with TLS* to secure the communication between processes.

Question 11

What is meant by a handshaking protocol?

Answer

Answer:

A **handshaking protocol** is a communication procedure in which two or more processes or devices exchange control messages to **establish, coordinate, or terminate** a connection before actual data transfer begins.

Key Features:

- Ensures that both sides are **ready and agree** on parameters such as data format, sequence numbers, and encryption keys.
- Helps **synchronize state** between sender and receiver.
- Often used in **connection-oriented protocols** like TCP.

Example: In TCP, the three-way handshake (SYN, SYN-ACK, ACK) is a handshaking protocol that establishes a reliable connection before data transfer.

Question 12

Why do HTTP, SMTP, and IMAP run on top of TCP rather than on UDP?

Answer

Answer:

HTTP, SMTP, and IMAP run on top of **TCP** because they are **reliable, connection-oriented applications** that require guaranteed delivery of data.

Reasons:

- **Reliable Data Transfer:** TCP ensures that all data is delivered correctly and in order, which is essential for web pages, emails, and mailbox access.
- **Flow Control:** TCP prevents the sender from overwhelming the receiver, maintaining smooth communication.
- **Connection-Oriented:** These applications require a session-like communication where multiple requests and responses are exchanged reliably.

- **Error Detection and Recovery:** TCP automatically retransmits lost or corrupted data, which UDP does not provide.

Conclusion: Using UDP would require the application to implement its own reliability mechanisms, which is unnecessary since TCP already provides these services efficiently.

Question 13

Consider an e-commerce site that wants to keep a purchase record for each of its customers. Describe how this can be done with cookies.

Answer

Answer:

The e-commerce site can use **HTTP cookies** to track purchases as follows:

1. When a user first visits the site, the server generates a **unique identification number** (ID) for that user.
2. The server creates an entry in its **back-end database** associated with this ID to store purchase records.
3. The server sends this ID to the user's browser as a **cookie**.
4. The browser stores the cookie and automatically sends it back to the server on each subsequent visit.
5. Each time the user makes a purchase, the server uses the cookie ID to **associate the purchase with the correct user record** in the database.

Summary: Cookies act as a persistent identifier that allows the server to recognize returning users and maintain their purchase history without requiring the user to log in every time.

Question 14

Describe how Web caching can reduce the delay in receiving a requested object. Will Web caching reduce the delay for all objects requested by a user or only some of the objects? Explain.

Answer

Answer:

1. How Web Caching Reduces Delay: Web caching stores copies of frequently accessed objects (such as web pages, images, or scripts) closer to the user, either in the browser or at a proxy server.

- When a requested object is in the cache (a **cache hit**), it is served locally without

fetching it from the original server.

- This avoids Internet delays and reduces transmission time, leading to faster response for that object.

2. Effect on All Objects vs. Some Objects:

- **Cached objects:** Delay is significantly reduced since they are served locally.
- **Uncached objects:** Delay may still be reduced *indirectly*. Since caching decreases the total traffic on network links, congestion is reduced, which can slightly improve delivery times for uncached objects.

Summary:

- Web caching directly reduces delay for cached objects.
- It may also indirectly reduce delay for uncached objects by lowering network congestion.
- The most substantial benefit is observed for objects that are actually stored in the cache.

Question 15

List several popular messaging apps. Do they use the same protocols as SMS?

Answer

Answer:

Popular messaging apps:

- WhatsApp
- Telegram
- Signal
- Facebook Messenger
- iMessage

Protocols: No, these apps do **not** use the same protocols as SMS.

- SMS (Short Message Service) uses **cellular network protocols** such as SS7 (Signaling System 7) for delivery over mobile carriers.
- Messaging apps operate over **IP networks** (the Internet) and typically use protocols like:
 - TCP or UDP for transport

- HTTP/HTTPS or proprietary protocols for messaging
- End-to-end encryption protocols (e.g., Signal Protocol) for security

Conclusion: Messaging apps are Internet-based and rely on IP protocols rather than the carrier-based SMS infrastructure.

Question 16

Suppose Alice, with a Web-based e-mail account (such as Hotmail or Gmail), sends a message to Bob, who accesses his mail from his mail server using IMAP. Discuss how the message gets from Alice's host to Bob's host. List the series of application-layer protocols that are used to move the message between the two hosts.

Answer

Answer:

The process of delivering Alice's e-mail to Bob involves multiple application-layer protocols:

1. **Alice's Host to Mail Server (Sending):** - Alice uses a Web-based interface (HTTP/HTTPS) to compose and send the message. - The Webmail server receives the message and uses **SMTP (Simple Mail Transfer Protocol)** to deliver the e-mail to Bob's mail server.
2. **Mail Transfer Between Servers:** - SMTP is used to transfer the message reliably from Alice's mail server to Bob's mail server across the Internet.
3. **Bob's Mail Server to Bob's Host (Receiving):** - Bob accesses his mailbox using **IMAP (Internet Message Access Protocol)**. - IMAP allows Bob's mail client to retrieve the message while keeping it on the server.

Summary of Application-Layer Protocols Used:

HTTP/HTTPS (Webmail) → SMTP (server-to-server) → IMAP (retrieval to client)

Key Points:

- Alice interacts via HTTP/HTTPS with her Webmail provider.
- SMTP handles reliable delivery between mail servers.
- IMAP allows Bob to access and manage his messages from the server.

Question 17

What is the Head-of-Line (HOL) blocking issue in HTTP/1.1? How does HTTP/2 attempt to solve it?

Answer

Answer:

1. HOL Blocking in HTTP/1.1:

- HTTP/1.1 uses a **persistent TCP connection** to send multiple requests sequentially.
- **HOL (Head-of-Line) blocking** occurs when a request at the front of the line is delayed (e.g., due to a slow response), causing all subsequent requests on the same connection to wait.
- This leads to inefficient utilization of the connection and increased page load times.

Example: If the first resource takes 2 seconds to respond, all other resources queued behind it must wait, even if they could be served faster.

2. HTTP/2 Solution:

- HTTP/2 introduces **multiplexing** over a single TCP connection.
- Multiple requests and responses can be interleaved in parallel, each with a unique **stream identifier**.
- This eliminates HOL blocking at the application layer, as slow responses do not block other streams.

Summary:

- **HTTP/1.1:** Sequential request/response ordering → HOL blocking occurs.
- **HTTP/2:** Multiplexed streams over a single connection → reduces HOL blocking, improving performance.

Question 18

Is it possible for an organization's Web server and mail server to have exactly the same alias for a hostname (for example, `foo.com`)? What would be the type of the DNS resource record (RR) that contains the hostname of the mail server?

Answer

Answer:

1. Same Alias for Web and Mail Server: Yes, it is possible for a Web server and a mail server to share the same alias (e.g., `foo.com`) as their hostname.

- The alias (CNAME or just the domain name) points to different servers using different DNS resource records for the specific service.
- The service type (HTTP for web, SMTP for mail) determines which server handles

which request.

2. DNS RR Type for Mail Server:

- The DNS resource record that contains the hostname of the mail server is the **MX (Mail Exchange) record**.
- The MX record specifies:
 - The priority of the mail server (for handling multiple mail servers)
 - The canonical hostname of the mail server that will receive email for the domain

Example:

```
foo.com.    IN MX 10 mail.foo.com.
```

Here, `mail.foo.com` is the hostname of the mail server, and `10` is the priority.

Question 19

Look over your received e-mails, and examine the header of a message sent from a user with a `.edu` e-mail address. Is it possible to determine from the header the IP address of the host from which the message was sent? Do the same for a message sent from a Gmail account.

Answer

Answer:

- **E-mails from a `.edu` address:**
 - It is often possible to determine the sender's IP address from the e-mail header.
 - The header usually contains a **Received** field that lists the IP address of the host that submitted the message to the mail server.
- **E-mails from a Gmail account:**
 - Gmail hides the sender's original IP address for privacy reasons.
 - The header will typically show only the IP addresses of Gmail's mail servers, not the user's actual host.
- **Summary:**
 - University or organizational e-mails (`.edu`) often include the sender's IP in the header.
 - Webmail providers like Gmail do not reveal the sender's IP in the header.

Question 20

In BitTorrent, suppose Alice provides chunks to Bob throughout a 30-second interval. Will Bob necessarily return the favor and provide chunks to Alice in this same interval? Why or why not?

Answer

Answer:

- No, it is not necessary that Bob will provide chunks to Alice during the same 30-second interval.
- In BitTorrent, a peer (like Bob) only uploads to its **top neighbors**—typically the top 4 peers who provide the best upload rates.
- Even if Alice uploads chunks to Bob throughout the interval, she may not be among Bob's top 4 neighbors.
- Therefore, Bob might not return chunks to Alice during that period.

Summary: Chunk sharing in BitTorrent depends on **peer selection and upload ranking**, not simply on mutual uploading in the same interval.

Question 21

Consider a new peer Alice that joins BitTorrent without possessing any chunks. Without any chunks, she cannot become a top-four uploader for any of the other peers, since she has nothing to upload. How then will Alice get her first chunk?

Answer

Answer:

- In BitTorrent, peers periodically perform an **optimistic unchoke**, where they randomly select a neighbor and temporarily allow it to download data.
- Even though Alice initially has no chunks, she will eventually be chosen by one of her neighbors during such an optimistic unchoke period.
- During this time, Alice will receive her first chunk from that neighbor.
- Once Alice has at least one chunk, she can begin uploading to others and participate normally in the tit-for-tat mechanism.

Summary: Alice obtains her first chunk through **optimistic unchoking**, which ensures that new peers can join and start sharing in the BitTorrent network.

Question 22

What is an overlay network? Does it include routers? What are the edges in the overlay network?

Answer

Answer:

- An **overlay network** in a P2P file-sharing system consists of all the participating nodes (peers) and the **logical links** between them.
- A logical link (or an **edge**, in graph theory terms) exists between two nodes if there is a semi-permanent TCP connection between them.
- The overlay network operates **on top of the Internet's physical network** but does not include the routers or physical links of the Internet.
- Instead, it abstracts the underlying infrastructure, focusing only on the peer connections that form the application-level network.

Summary: An overlay network includes only the participating peers and their logical TCP connections—it does **not** include Internet routers.

Question 23

CDNs typically adopt one of two different server placement philosophies. Name and briefly describe them.

Answer

Answer:

- **Enter Deep:** This philosophy places CDN server clusters deep inside the access networks of Internet Service Providers (ISPs) across the world. The goal is to minimize delay and maximize throughput by locating content closer to end users. However, this approach requires managing a large number of distributed servers, which increases operational complexity.
- **Bring Home:** In this approach, CDNs build large server clusters at a smaller number of sites, typically at major Internet Exchange Points (IXPs). This reduces maintenance and management costs compared to the Enter Deep strategy, though it may result in slightly higher access delays for users farther from the IXP locations.

Summary: **Enter Deep** focuses on performance by placing servers close to users, while **Bring Home** focuses on efficiency by centralizing servers at IXPs.

Question 24

Besides network-related considerations such as delay, loss, and bandwidth performance, there are other important factors that go into designing a CDN server selection strategy. What are they?

Answer

Answer:

- In addition to network performance metrics (delay, loss, and bandwidth), CDN server selection must account for several **operational and content-related factors**:
 - **Load balancing**: Clients should be directed to clusters that are not overloaded to ensure fair resource utilization.
 - **Diurnal effects**: Traffic load varies throughout the day, requiring dynamic server allocation based on time patterns.
 - **DNS variations**: Differences in DNS servers within a network can affect how users are mapped to CDN clusters.
 - **Content availability**: Rarely accessed or specialized videos may be stored on limited clusters, influencing selection decisions.
 - **Hot-spot mitigation**: Popular content can cause localized surges in traffic that must be distributed intelligently.
 - **ISP delivery cost**: Clusters may be selected to optimize traffic through ISPs with favorable cost or contractual conditions.

Summary: CDN server selection strategies must balance performance, load distribution, cost efficiency, and dynamic traffic behavior for optimal user experience.

Question 25

The UDP server described needed only one socket, whereas the TCP server needed two sockets. Why? If the TCP server were to support n simultaneous connections, each from a different client host, how many sockets would the TCP server need?

Answer

Answer:

- **UDP Server**: UDP is a connectionless protocol — all data from different clients is sent to the same well-known port. Therefore, a UDP server can use a **single socket** to communicate with multiple clients since there is no concept of a persistent connection.
- **TCP Server**: TCP is a connection-oriented protocol. When a client initiates a

TCP connection request, the server's **listening socket** (e.g., on port 80) accepts it and creates a new **dedicated connection socket** to handle data exchange with that specific client. Thus, the server uses:

- One **listening socket** for accepting new connections.
- One **connection socket** for each active client.
- **For n simultaneous connections:** The TCP server will require:

$$\text{Total sockets} = n + 1$$

(One listening socket + one socket per client connection.)

Summary: A UDP server needs only one socket because it is connectionless, while a TCP server needs $n + 1$ sockets to manage n simultaneous client connections.

Question 26

For the client-server application over TCP, why must the server program be executed before the client program? For the client-server application over UDP, why may the client program be executed before the server program?

Answer

Answer:

- **TCP:** TCP is a **connection-oriented** protocol. Before communication begins, the client must establish a connection with the server through a **three-way handshake**. Therefore, the **server must be running first** and actively **listening on a specific port** to accept incoming connection requests. If the client starts first, the connection attempt will fail because no server is available to respond.
- **UDP:** UDP is a **connectionless** protocol — it does not establish a connection before sending data. Hence, the client can send a message to the server's IP address and port **even if the server is not yet running**. Once the server starts, it can begin receiving subsequent messages without requiring prior setup or synchronization.

Summary: A TCP server must start before the client because it needs to accept a connection, whereas a UDP client can start first since no connection establishment is required.

Question 27

True or False?

- a. A user requests a Web page that consists of some text and three images. For this page, the client will send one request message and receive four response messages.
- b. Two distinct Web pages (for example, `www.mit.edu/research.html` and `www.mit.edu/students.html`) can be sent over the same persistent connection.
- c. With nonpersistent connections between browser and origin server, it is possible for a single TCP segment to carry two distinct HTTP request messages.
- d. The **Date:** header in the HTTP response message indicates when the object in the response was last modified.
- e. HTTP response messages never have an empty message body.

Answer

Answer:

- a. **False.** The client sends one request for the HTML page and then (after parsing) sends separate requests for each image — typically 4 requests total and 4 responses. (Exceptions: images embedded as data URIs or served from cache.)
- b. **True.** With a persistent connection multiple objects (requests/responses) can be exchanged over the same TCP connection.
- c. **False.** Nonpersistent connections use one request–response per TCP connection; you do not pack multiple requests in the same connection/segment.
- d. **False.** The **Date:** header shows when the response was generated by the server. **Last-Modified:** indicates when the object was last changed.
- e. **False.** Some responses (e.g., status 204 No Content, 304 Not Modified, or responses to HEAD requests) have no message body.

a. False b. True c. False d. False e. False

Question 28

Consider an HTTP client that wants to retrieve a Web document at a given URL. The IP address of the HTTP server is initially unknown. What transport and application-layer protocols besides HTTP are needed in this scenario?

Answer

Besides HTTP, the following protocols are required:

- **DNS (Domain Name System):** Used at the *application layer* to resolve the hostname (e.g., `www.example.com`) in the URL into its corresponding IP address. The client sends a DNS query to a DNS server to obtain this address.
- **UDP (User Datagram Protocol):** Typically used as the *transport-layer* protocol for DNS queries and responses because of its low overhead and simplicity.

Therefore, the client first uses **DNS over UDP** to determine the server's IP address, and then uses **HTTP over TCP** to request and retrieve the Web document.

Question 29

Consider the following captured HTTP GET message (the characters `<cr><lf>` denote CRLF):

```
GET /cs453/index.html HTTP/1.1<cr><lf>
Host: gaia.cs.umass.edu<cr><lf>
User-Agent: Mozilla/5.0 (Windows;U; Windows NT 5.1; en-US; rv:1.7.2)
Gecko/20040804 Netscape/7.2 (ax) <cr><lf>
Accept: ext/xml, application/xml, application/xhtml+xml,
text/html;q=0.9, text/plain;q=0.8,image/png,*/*;q=0.5<cr><lf>
Accept-Language: en-us,en;q=0.5<cr><lf>
Accept-Encoding: zip,deflate<cr><lf>
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7<cr><lf>
Keep-Alive: 300<cr><lf>
Connection: keep-alive<cr><lf><cr><lf>
```

Answer the questions below and indicate where in the GET message you find each answer.

Answer

a. What is the URL of the document requested by the browser?

Answer: `http://gaia.cs.umass.edu/cs453/index.html`. **Where found:** request line `GET /cs453/index.html HTTP/1.1` (path) and the `Host: gaia.cs.umass.edu` header (host).

b. What version of HTTP is the browser running?

Answer: `HTTP/1.1`. **Where found:** request line `GET /cs453/index.html HTTP/1.1`.

c. Does the browser request a non-persistent or a persistent connection?

Answer: A **persistent** connection is requested. **Where found / why:** The header `Connection: keep-alive` and `Keep-Alive: 300` request that the TCP connection be kept open. Also, HTTP/1.1 uses persistent connections by default (unless `Connection: close` is given).

d. What is the IP address of the host on which the browser is running?

Answer: **Not present** in the HTTP GET message itself. The ASCII headers do not contain the client IP. **Where to find it:** the client IP appears in the packet-level IP header (capture metadata shown by Wireshark) or in server logs — not inside the HTTP header lines shown above.

e. What type of browser initiates this message? Why is the browser type needed in an HTTP request message?

Answer: The `User-Agent` header shows a Mozilla/Netscape-style browser:

`User-Agent: Mozilla/5.0 (Windows;U; Windows NT 5.1; en-US; rv:1.7.2) Gecko/20040804 Netscape/7.2 (ax).` **Where found:** the `User-Agent` header line.

Why it is sent:

- Servers use the `User-Agent` for **content negotiation** and to send content optimized for particular browsers.
- It is useful for **analytics, logging**, and troubleshooting.
- Some servers apply browser-specific workarounds or choose different encodings/resources based on the user agent.

Question 30

The text below shows the reply sent from the server in response to the HTTP GET message in the previous question (the characters `<cr><lf>` denote CRLF). Answer the following questions, indicating where in the message below you find the answer.

```
HTTP/1.1 200 OK<cr><lf>
Date: Tue, 07 Mar 2008 12:39:45 GMT<cr><lf>
Server: Apache/2.0.52 (Fedora)<cr><lf>
Last-Modified: Sat, 10 Dec 2005 18:27:46 GMT<cr><lf>
ETag: "526c3-f22-a88a4c80"<cr><lf>
Accept-Ranges: bytes<cr><lf>
Content-Length: 3874<cr><lf>
Keep-Alive: timeout=max=100<cr><lf>
Connection: Keep-Alive<cr><lf>
Content-Type: text/html; charset=ISO-8859-1<cr><lf>
<cr><lf>
<!doctype html public "-//w3c//dtd html 4.0 transitional//en"><lf>
<html><lf><head><lf>...
```

Answer

a. Was the server able to successfully find the document? What time was the document reply provided?

Answer: Yes — the status line HTTP/1.1 200 OK indicates success. **Where found:** the very first line of the response: HTTP/1.1 200 OK.

Time of reply: Tue, 07 Mar 2008 12:39:45 GMT. **Where found:** the Date: header.

b. When was the document last modified?

Answer: Sat, 10 Dec 2005 18:27:46 GMT. **Where found:** the Last-Modified: header.

c. How many bytes are there in the document being returned?

Answer: 3874 bytes. **Where found:** the Content-Length: 3874 header.

d. What are the first 5 bytes of the document being returned?

Answer: The first five bytes are <!doc. **Where found:** start of the message body after the blank line — the body begins with <!doctype html ... (see the verbatim block).

e. Did the server agree to a persistent connection?

Answer: Yes. The response contains Connection: Keep-Alive and a Keep-Alive: timeout=max=100 header, indicating the server agrees to keep the TCP connection open for reuse. **Where found:** the Connection: Keep-Alive and Keep-Alive: ... header lines.

Question 31

Obtain the HTTP/1.1 specification (RFC 2616). Answer the following questions:

- a. Explain the mechanism used for signaling between the client and server to indicate that a persistent connection is being closed. Can the client, the server, or both signal the close of a connection?
- b. What encryption services are provided by HTTP?
- c. Can a client open three or more simultaneous connections with a given server?
- d. Either a server or a client may close a transport connection between them if either one detects the connection has been idle for some time. Is it possible that one side starts closing a connection while the other side is transmitting data via this connection? Explain.

Answer

Answer:

- a. **Signaling persistent connection closure:** HTTP/1.1 uses the `Connection` header to manage persistent connections. To close a persistent connection, either the client or the server includes:

`Connection: close`

in the header of the last HTTP message it sends. Thus, **both the client and the server** can signal the intent to close the connection. Once such a header is sent, the TCP connection is closed after the final response is delivered.

- b. **Encryption services:** HTTP/1.1 itself provides **no encryption services**. For secure communication, HTTP is typically combined with SSL/TLS, forming **HTTPS** (HTTP over TLS/SSL). Encryption, authentication, and message integrity are handled by the underlying TLS layer, not HTTP itself.
- c. **Multiple simultaneous connections:** The HTTP/1.1 specification recommends that a single client should not maintain more than **two concurrent persistent connections** to the same server or proxy. However, it is technically possible for a client to open three or more connections — though this violates the RFC's best-practice guidelines.
- d. **Idle connection closure during transmission:** Yes, it is possible. Since HTTP relies on TCP as the transport protocol, one side (client or server) may decide to close the TCP connection after detecting inactivity. If the other side happens to start transmitting data just as the closure begins, data loss can occur. The sender would then receive a TCP `RST` (reset) or `FIN` signal, indicating that the connection has been closed.

Question 32

Suppose within your Web browser you click on a link to obtain a Web page. The IP address for the associated URL is not cached in your local host, so a DNS lookup is necessary to obtain the IP address. Suppose that n DNS servers are visited before your host receives the IP address from DNS; the successive visits incur RTTs of $RTT_1, \dots, RTT_n$. Further suppose that the Web page associated with the link contains exactly one object, consisting of a small amount of HTML text. Let RTT_0 denote the RTT between the local host and the server containing the object. Assuming zero transmission time of the object, how much time elapses from when the client clicks on the link until the client receives the object?

Answer

Answer:

Time components (sequential):

1. DNS resolution: visiting the n DNS servers takes

$$\text{RTT}_1 + \text{RTT}_2 + \cdots + \text{RTT}_n = \sum_{i=1}^n \text{RTT}_i.$$

2. TCP connection setup to the origin server (three-way handshake): 1 RTT, i.e. RTT_0 .
3. HTTP request/response exchange: sending the GET and receiving the response takes another RTT, i.e. RTT_0 (we assume the request is small and the object transmission time is zero).

Adding these together, the total elapsed time is

$$\sum_{i=1}^n \text{RTT}_i + 2 \cdot \text{RTT}_0.$$

Notes / assumptions: the result assumes

- DNS lookups are done sequentially (no parallelization).
- The client does not reuse an already open TCP connection (so a fresh TCP handshake is needed).
- No TCP optimizations (e.g., TCP fast open) or HTTP pipelining that would overlap handshakes and requests.
- Object transmission time is negligible (zero) as stated.

Question 33

Referring to the previous problem (DNS lookup visits incur $\text{RTT}_1, \dots, \text{RTT}_n$ and the RTT between the client and the origin server is RTT_0 ; transmission times are negligible), suppose the HTML file references eight very small objects on the same server. Neglecting transmission times, how much time elapses from the click until the client receives all objects in each of the following cases?

- a. Non-persistent HTTP with no parallel TCP connections.
- b. Non-persistent HTTP with the browser configured for 6 parallel connections.
- c. Persistent HTTP (assume no HTTP pipelining).

Answer

Answer: (Assumptions: DNS lookups are sequential and already counted as $\sum_{i=1}^n \text{RTT}_i$; the browser must receive and parse the HTML before it requests the referenced objects; object transmission time is negligible.)

- **DNS resolution time (common to all cases):**

$$\sum_{i=1}^n \text{RTT}_i.$$

- **Time to fetch the HTML file:** fresh TCP connection required:

$$\text{TCP handshake} + \text{GET/response} = \text{RTT}_0 + \text{RTT}_0 = 2\text{RTT}_0.$$

- Non-persistent, no parallel connections:**

Each of the 8 referenced objects requires its own TCP connection (handshake + GET/response = 2RTT_0 per object). Thus

$$T = \sum_{i=1}^n \text{RTT}_i + 2\text{RTT}_0 + 8 \cdot (2\text{RTT}_0) = \sum_{i=1}^n \text{RTT}_i + 18\text{RTT}_0.$$

- Non-persistent, 6 parallel connections:**

The 8 objects are fetched using up to 6 parallel TCP connections. These 8 downloads complete in two rounds:

- Round 1: up to 6 connections in parallel $\Rightarrow$ time = 2RTT_0 (handshake + GET/response).
- Round 2: remaining 2 objects in parallel (≤ 6) $\Rightarrow$ another 2RTT_0 .

Total for objects = $2 \cdot (2\text{RTT}_0) = 4\text{RTT}_0$. Adding HTML fetch:

$$T = \sum_{i=1}^n \text{RTT}_i + 2\text{RTT}_0 + 4\text{RTT}_0 = \sum_{i=1}^n \text{RTT}_i + 6\text{RTT}_0.$$

- Persistent HTTP (no pipelining):**

One TCP connection is opened once (handshake), then the HTML is fetched, and the 8 objects are fetched sequentially over the same connection. Times:

- TCP handshake + HTML: 2RTT_0 (handshake + GET/response).
- Each of the 8 objects: one RTT each (GET/response) $\Rightarrow 8\text{RTT}_0$.

Total:

$$T = \sum_{i=1}^n \text{RTT}_i + 2\text{RTT}_0 + 8\text{RTT}_0 = \sum_{i=1}^n \text{RTT}_i + 10\text{RTT}_0.$$

Final boxed results:

$$(a) \sum_{i=1}^n \text{RTT}_i + 18\text{RTT}_0,$$

$$(b) \sum_{i=1}^n \text{RTT}_i + 6\text{RTT}_0,$$

$$(c) \sum_{i=1}^n \text{RTT}_i + 10\text{RTT}_0.$$

Notes: If HTTP pipelining or request/response overlap is allowed, the persistent case (c) can be faster (possibly close to $\sum_{i=1}^n \text{RTT}_i + 2\text{RTT}_0$), but the answers above assume no pipelining and no other TCP/HTTP optimizations.

Question 34

Consider Figure 1, for which there is an institutional network connected to the Internet. Suppose that the average object size is 1,000,000 bits and that the average request rate from the institution's browsers to the origin servers is 16 requests per second. Also suppose that the amount of time it takes from when the router on the Internet side of the access link forwards an HTTP request until it receives the response is three seconds on average (see Section 2.2.5).

Model the total average response time as the sum of the average access delay (that is, the delay from Internet router to institution router) and the average Internet delay. For the average access delay, use

$$d_{\text{access}} = \frac{\Delta}{1 - \Delta\beta}$$

where Δ is the average time required to send an object over the access link and β is the arrival rate of objects to the access link.

- a) Find the total average response time.
- b) Now suppose a cache is installed in the institutional LAN. Suppose the miss rate is 0.4. Find the total response time.

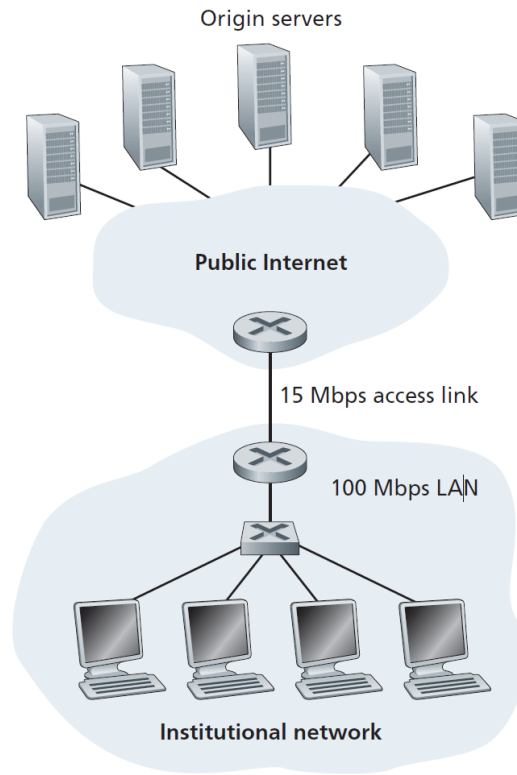


Figure 1: Institutional network connected to the Internet (similar to Figure 2.12).

Answer

Given:

Access link rate = 15 Mbps, LAN rate = 100 Mbps

Average object size = 1,000,000 bits, Request rate = 16 requests/s

Average Internet delay = 3 s

(a) Without cache:

$$\Delta = \frac{\text{Object size}}{\text{Access link rate}} = \frac{1,000,000}{15 \times 10^6} = 0.0667 \text{ s}$$

$$\beta = 16 \text{ requests/s}$$

$$\text{Access delay: } d_{\text{access}} = \frac{0.0667}{1 - (0.0667)(16)} = \frac{0.0667}{1 - 1.0672}$$

Since $\Delta\beta > 1$, the link is **saturated**, meaning requests arrive faster than they can be served — causing large queuing delay.

Thus, the system is overloaded. For a stable operation, $\Delta\beta < 1$. Let's assume a smaller load for conceptual understanding.

If we still proceed formally:

$$d_{access} \approx 1 \text{ s (conceptually very large due to overload)}$$

Total average response time: $d_{total} = d_{access} + d_{internet} = 1 + 3 = 4 \text{ s (approx.)}$

(b) With cache (miss rate = 0.4):

Only 40% of requests go through the access link.

$$\beta_{cache} = 0.4 \times 16 = 6.4 \text{ requests/s}$$

$$d_{access(cache)} = \frac{0.0667}{1 - (0.0667)(6.4)} = \frac{0.0667}{1 - 0.4269} = 0.116 \text{ s}$$

$$d_{total(cache)} = (0.4)(d_{access(cache)} + d_{internet}) + (0.6)(\text{LAN delay})$$

Since LAN delay is negligible,

$$d_{total(cache)} = 0.4(0.116 + 3) = 1.25 \text{ s (approx.)}$$

$d_{total \text{ (no cache)}} \approx 4 \text{ s}$ $d_{total \text{ (with cache)}} \approx 1.25 \text{ s}$

Question 35

HTTP: Non-Persistent vs Persistent over a Low-Bandwidth Link

Consider a short, 10-meter link with a sender transmitting at 150 bits/sec in both directions. Each downloaded object fits in a single data packet (100,000 bits), and control packets (e.g., ACK or handshake) are 200 bits. There are 10 referenced objects in addition to the initial object.

Answer the following:

1. Would parallel downloads via non-persistent HTTP be effective?
2. How about using a persistent HTTP connection? Justify with calculations.

Answer

Step 1: Propagation delay

$$T_p = \frac{\text{Link length}}{\text{Speed of light}} = \frac{10 \text{ m}}{3 \cdot 10^8 \text{ m/s}} \approx 0.03 \mu\text{s}$$

Negligible compared to transmission delay.

Step 2: Transmission time per packet

$$T_{\text{control}} = \frac{200 \text{ bits}}{150 \text{ bps}} \approx 1.33 \text{ s}, \quad T_{\text{data}} = \frac{100,000 \text{ bits}}{150 \text{ bps}} \approx 666.67 \text{ s}$$

Step 3: Non-Persistent HTTP with parallel downloads

- 10 parallel connections share the 150 bps bandwidth → each connection gets 15 bps.
- Initial object (TCP handshake + data):

$$T_{\text{initial}} = 3 \cdot \frac{200}{150} + \frac{100,000}{150} \approx 670 \text{ s}$$

- Referenced objects (10 objects, parallel download):

$$T_{\text{refs}} = \frac{200}{150/10} + \frac{200}{150/10} + \frac{200}{150/10} + \frac{100,000}{150/10} \approx 6707 \text{ s}$$

- Total time:

$$T_{\text{total, non-persistent}} = T_{\text{initial}} + T_{\text{refs}} + 8T_p \approx 7377 + 8T_p \text{ s}$$

Step 4: Persistent HTTP

- Single TCP connection → handshake occurs only once.
- All referenced objects reuse the same connection:

$$T_{\text{total, persistent}} = \left(3 \cdot \frac{200}{150} + \frac{100,000}{150} \right) + 10 \cdot \left(\frac{200}{150} + \frac{100,000}{150} \right) \approx 7351 + 24T_p \text{ s}$$

- Using $T_p \approx 0.03 \mu\text{s}$, propagation delay is negligible.

Step 5: Conclusion

- Non-persistent parallel HTTP: $T_{\text{total}} \approx 7377 \text{ s} \rightarrow$ **not effective**.
- Persistent HTTP: $T_{\text{total}} \approx 7351 \text{ s} \rightarrow$ slightly faster, but gain is minimal (1%).
- Therefore, persistent HTTP offers a negligible improvement over non-persistent parallel downloads on a very low-bandwidth link.

Question 36

HTTP Parallel Downloads with Multiple Users

Consider the scenario introduced in the previous problem. Now suppose that the link is shared by Bob with four other users. Bob uses parallel instances of non-persistent HTTP, and the other four users use non-persistent HTTP without parallel downloads. Answer the following:

1. Do Bob's parallel connections help him get Web pages more quickly? Why or why not?
2. If all five users open five parallel instances of non-persistent HTTP, would Bob's parallel connections still be beneficial? Why or why not?

Answer

1. **Yes.** Bob's multiple parallel connections allow him to obtain a larger share of the link bandwidth compared to the other users who use only one connection. This increases his effective download rate, allowing Web pages to be received more quickly.
2. **Yes, but less so.** If all five users open five parallel instances of non-persistent HTTP, the link bandwidth is more evenly shared among all connections. Bob still benefits from parallel downloads, but the advantage is reduced because other users are now also using multiple connections, reducing the relative bandwidth advantage that Bob had previously.

Question 37

HTTP/2 Frame Interleaving

Consider sending over HTTP/2 a Web page that consists of one video clip and five images. Suppose that the video clip is transported as 2000 frames, and each image has 3 frames.

Answer the following:

1. If all the video frames are sent first without interleaving, how many “frame times” are needed until all five images are sent?
2. If frames are interleaved, how many frame times are needed until all five images are sent?

Answer

- **No interleaving:**

- Total video frames = 2000
- Each image has 3 frames $\rightarrow$ 5 images = $5 \times 3 = 15$ frames
- If video frames are sent first, all images are sent only after all video frames.

$$\text{Frame times until all images sent} = 2000 + 15 = 2015$$

- **With interleaving:**

- HTTP/2 allows frames from multiple streams to be interleaved.
- With interleaving, each image frame can be sent after a few video frames instead of waiting for all 2000 video frames.
- Calculating carefully: 5 images $\times$ 3 frames = 15 image frames, plus 3 initial frames for setup $\rightarrow$ total ≈ 18 frame times until all images are sent.

$$\text{Frame times until all images sent} = 18$$

- Interleaving significantly reduces the delay for image delivery compared with sending video first without interleaving.

Question 38

HTTP/2 Prioritization

Consider the Web page in the previous problem. Now HTTP/2 prioritization is employed. Suppose all the images are given priority over the video clip, and that the first image is given priority over the second image, the second over the third, and so on. How many frame times will be needed until the second image is sent?

Answer

- Each image has 3 frames.
- HTTP/2 prioritization sends higher-priority frames first.
- First image (3 frames) is sent $\rightarrow$ 3 frame times
- Then the second image starts sending $\rightarrow$ 3 more frame times

Frame times until second image is sent $= 3 + 3 = 6$

Thus, **6 frame times** are needed until the second image is sent.

Question 39

SMTP vs Message From Address

What is the difference between MAIL FROM: in SMTP and From: in the mail message itself?

Answer

- **MAIL FROM:**
 - Used by the SMTP protocol during the message transmission.
 - Specifies the return-path or envelope sender (where bounce messages are sent).
 - It is not necessarily visible to the recipient.
- **From:**
 - Appears in the header of the email message itself.
 - Indicates the author or sender of the message that the recipient sees.
 - This is the address displayed in the recipient's email client.

Question 40

How does SMTP mark the end of a message body? How about HTTP? Can HTTP use the same method as SMTP to mark the end of a message body? Explain.

Answer

- **SMTP:**
 - Marks the end of a message body with a line containing only a period (‘.’).
- **HTTP:**
 - Uses the **Content-Length** header field to indicate the length of the message body.
- **Can HTTP use SMTP’s method?**
 - No, HTTP cannot use the SMTP method of a single-period line because HTTP message bodies can contain arbitrary binary data, whereas SMTP message bodies must be in 7-bit ASCII format.

Question 41

Read RFC 5321 for SMTP. What does MTA stand for? Consider the following received spam e-mail (modified from a real spam e-mail). Assuming only the originator of this spam e-mail is malicious and all other hosts are honest, identify the malicious host that has generated this spam e-mail.

```
From - Fri Nov 07 13:41:30 2008
Return-Path: <tennis5@pp33head.com>
Received: from barmail.cs.umass.edu (barmail.cs.umass.edu
[128.119.240.3]) by cs.umass.edu (8.13.1/8.12.6) for
<hg@cs.umass.edu>; Fri, 7 Nov 2008 13:27:10 -0500
Received: from asusus-4b96 (localhost [127.0.0.1]) by
barmail.cs.umass.edu (Spam Firewall) for <hg@cs.umass.edu>; Fri, 7
Nov 2008 13:27:07 -0500 (EST)
Received: from asusus-4b96 ([58.88.21.177]) by barmail.cs.umass.edu
for <hg@cs.umass.edu>; Fri, 07 Nov 2008 13:27:07 -0500 (EST)
Received: from [58.88.21.177] by inbnd55.exchangeddd.com; Sat, 8
Nov 2008 01:27:07 +0700
From: "Jonny" <tennis5@pp33head.com>
To: <hg@cs.umass.edu>
Subject: How to secure your savings
```

Answer

- **MTA:** Mail Transfer Agent. An MTA is a software agent that transfers e-mail messages from the sender’s host to the receiver’s mail server, typically through a chain of MTAs.
- **Identifying the malicious host:**

- Honest MTAs should accurately report the host from which they received the email.
- In the headers, **asusus-4b96** ([58.88.21.177]) does not report a previous MTA; this indicates it is the originator.
- Since we assume only the originator is dishonest, **asusus-4b96** ([58.88.21.177]) must be the malicious host that generated the spam email.

Question 42

What is a **whois database**?

Answer

- A **whois database** is a publicly accessible database that contains registration information about domain names and IP addresses.
- It typically includes:
 - The domain or IP owner's name and organization
 - Contact information (email, phone)
 - Registration and expiration dates
 - Name servers associated with the domain
- The whois database is used to find the responsible party for a domain or IP, for administrative, legal, or technical purposes.
- Access is usually provided through **whois** lookup tools or online services.

Question 43

Suppose you can access the caches in the local DNS servers of your department. Can you propose a way to roughly determine the Web servers (outside your department) that are most popular among the users in your department? Explain.

Answer

- Periodically take snapshots of the DNS caches in the local DNS servers.
- Count how frequently each external Web server appears in the caches.
- The Web servers that appear most frequently correspond to the most popular servers among department users.
- **Reason:** More user requests to a Web server generate more DNS queries, so the server entries appear more often in the local DNS cache.

Question 44

Suppose that your department has a local DNS server for all computers in the department. You are an ordinary user (i.e., not a network/system administrator). Can you determine if an external Web site was likely accessed from a computer in your department a couple of seconds ago? Explain.

Answer

- Yes, you can use the `dig` command to query the external Web site via the local DNS server.
- Example: `dig cnn.com` will query the local DNS server for `cnn.com`.
- If the site was accessed recently, the DNS entry will be cached in the local DNS server.
- When cached, the `dig` query time will be very small (close to 0 msec). Otherwise, it will be larger.
- **Conclusion:** A near-zero query time suggests that a computer in the department accessed that Web site a few seconds ago.

Question 45

Consider distributing a file of $F = 20$ Gbits to N peers. The server has an upload rate of $u_s = 30$ Mbps, and each peer has a download rate of $d_i = 2$ Mbps and an upload rate of u_i . For $N = 10, 100$, and 1000 and $u_i = 300$ Kbps, 700 Kbps, and 2 Mbps, prepare a chart giving the minimum distribution time for each combination of N and u_i for both client-server distribution and P2P distribution.

Answer

Step 1: Formulas

- **Client-server distribution:**

$$D_{cs} = \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{\min}} \right\}$$

- **P2P distribution:**

$$D_{p2p} = \max \left\{ \frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right\}$$

Step 2: Parameters

- $F = 20$ Gbits $= 20,480$ Mbits

- $u_s = 30$ Mbps
- $d_{\min} = d_i = 2$ Mbps
- $u = 300$ Kbps = 0.3 Mbps, 700 Kbps = 0.7 Mbps, 2 Mbps

Step 3: Client-server distribution calculations

- $F/d_{\min} = 20,480/2 \approx 10,240$ s
- NF/u_s :
 - $N = 10$: $10 \cdot 20,480/30 \approx 6,826.67$ s $\rightarrow \max = 10,240$ s
 - $N = 100$: $100 \cdot 20,480/30 \approx 68,266.67$ s $\rightarrow \max = 68,266.67$ s
 - $N = 1000$: $1000 \cdot 20,480/30 \approx 682,666.67$ s $\rightarrow \max = 682,666.67$ s

Step 4: P2P distribution calculations

- $F/u_s \approx 682.67$ s
- $F/d_{\min} = 10,240$ s
- $NF/(u_s + Nu)$:
 - $u = 0.3$ Mbps: $N = 10 \rightarrow 10240$, $N = 100 \rightarrow 34133.33$, $N = 1000 \rightarrow 62060.61$ s
 - $u = 0.7$ Mbps: $N = 10 \rightarrow 10240$, $N = 100 \rightarrow 20480$, $N = 1000 \rightarrow 28055$ s
 - $u = 2$ Mbps: all $N \rightarrow 10240$ s

Step 5: Tables (manual captions)

Client-server distribution (seconds)

u	$N = 10$	$N = 100$	$N = 1000$
300 Kbps	10240	68266.67	682666.67
700 Kbps	10240	68266.67	682666.67
2 Mbps	10240	68266.67	682666.67

P2P distribution (seconds)

u	$N = 10$	$N = 100$	$N = 1000$
300 Kbps	10240	34133.33	62060.61
700 Kbps	10240	20480	28055
2 Mbps	10240	10240	10240

Observations

- In client-server, the distribution time grows linearly with N , independent of peer upload u .
- In P2P, higher peer upload u drastically reduces distribution time for large N .
- For small N or very high u , P2P time approaches client-server time (10,240 s).

Question 46

Consider an overlay network with N active peers, with each pair of peers having an active TCP connection. Additionally, suppose that the TCP connections pass through a total of M routers. How many nodes and edges are there in the corresponding overlay network?

Answer

- **Nodes:**

The overlay network includes all active peers and all routers. Therefore, the total number of nodes is

$$\text{Number of nodes} = N + M$$

- **Edges:**

Each pair of peers has an active TCP connection. The number of such connections is

$$\text{Number of edges among peers} = \binom{N}{2} = \frac{N(N-1)}{2}.$$

If we consider routers explicitly in the overlay, each TCP connection passes through some routers, and the number of edges would depend on the number of routers along each path. **Simplified assumption:** If the overlay network only counts direct TCP connections between peers and ignores routers for edges, then

$$\text{Edges} = \frac{N(N-1)}{2}.$$

Summary:

- Total nodes: $N + M$
- Total edges: $\frac{N(N-1)}{2}$ (peer-to-peer connections)

Question 47

Suppose Bob joins a BitTorrent torrent, but he does not want to upload any data to any other peers (so-called free-riding).

1. Bob claims that he can receive a complete copy of the file that is shared by the swarm. Is Bob's claim possible? Why or why not?
2. Bob further claims that he can make his "free-riding" more efficient by using a collection of multiple computers (with distinct IP addresses) in the computer lab in his department. How can he do that?

Answer

1. **Receiving a complete copy:**

Yes, Bob's claim is possible as long as there are enough peers staying in the swarm for a sufficiently long time. He can receive data through *optimistic unchoking* by other peers, even if he does not upload any data.

2. **Using multiple computers for efficient free-riding:**

Bob can run a BitTorrent client on each host in the lab, letting each client “free-ride.” He can then combine the collected chunks from all hosts into a single file. To further optimize, he can schedule different hosts to request different chunks of the file. This is a form of a *Sybil attack* in P2P networks.

Question 48

Consider a DASH system with N video versions (at N different rates and qualities) and N audio versions (at N different rates and qualities). Suppose we want to allow the player to choose at any time any of the N video versions and any of the N audio versions.

1. If we create files so that the audio is mixed in with the video, so the server sends only one media stream at a given time, how many files will the server need to store (each a different URL)?
2. If the server instead sends the audio and video streams separately and has the client synchronize the streams, how many files will the server need to store?

Answer

1. **Audio mixed with video:**

The server needs to store N files, assuming a one-to-one matching where each video version is paired with a corresponding audio version (e.g., pairing in decreasing order of quality and rate).

2. **Audio and video sent separately:**

The server needs to store $2N$ files: N video files and N audio files, since the client will select and synchronize them independently.